Gnanamani College of Technology
(AUTONOMOUS)
Approved by AICTE, New Delhi & Affiliated to Anna University, Chennai
NH-7, A.K. Samuthiram, Pachal – 637018, Namakkal (Dt.), Tamilnadu, India.
Web : www.gct.org.in, E-Mail : info@gct.org.in, Contact No. 7598293888, 7598293999

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & DATA SCIENCE

23AD522 – DATA AND INFORMATION SECURITY LAB

3 0 0 3 3

Course Objectives:

The main learning objective of this course is to prepare the students for:

- To learn different cipher techniques
- To implement the algorithms DES, RSA
- To implement the algorithms MD5, SHA-1
- To use network security tools
- To use network vulnerability assessment tools

LIST OF EXPERIMENTS

1. Perform encryption, decryption using the following substitution techniques
(i) Ceaser cipher, (ii) playfair cipher iii) Hill Cipher iv) Vigenere cipher
2. Perform encryption and decryption using following transposition techniques
i) Rail fence ii) row & Column Transformation
3. Apply DES algorithm for practical applications.
4. Apply AES algorithm for practical applications.
5. Implement RSA Algorithm using HTML and JavaScript
6. Implement the Diffie-Hellman Key Exchange algorithm for a given problem.
7. Calculate the message digest of a text using the SHA-1 algorithm.
8. Implement the SIGNATURE SCHEME - Digital Signature Standard.
9. Demonstrate intrusion detection system (ids) using any tool eg. Snort or any other s/w.
10. Automated Attack and Penetration Tools
    Exploring N-Stalker, a Vulnerability Assessment Tool
11. Defeating Malware
    i) Building Trojans ii) Rootkit Hunter

Total=45 Periods

Lab Requirements:

S.No	Description of Equipment	Required Numbers(for batch of 30 students)
1	INTEL based desktop PC with min. 8GB RAM and 500 GB HDD, 17” or higher TFT Monitor, Keyboard and mouse	30
2	Windows 10 or higher operating system / Linux Ubuntu 20 or higher	30
3	C / C++ / Java or equivalent compiler GnuPG, Snort, N-Stalker or Equivalent.	30

Total = 45 Periods

Course Outcomes:

Students will be able:

CO1 To develop code for classical Encryption Techniques to solve the problems

CO2 To build cryptosystems by applying symmetric and public key encryption algorithms

CO3 To construct code for authentication algorithms.

CO4 To develop a signature scheme using Digital signature standard

CO5 To demonstrate the network security system using open source tools

COs vs POs Mapping:

CO's	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2
CO1	3	2	3	1	-	-	-	-	1	3	1	2	3	1
CO2	1	3	3	3	2	-	-	-	1	2	2	2	1	2
CO3	2	3	3	3	1	-	-	-	1	3	1	2	1	2
CO4	3	3	1	1	1	-	-	-	3	1	1	3	2	3
CO5	3	2	2	3	2	-	-	-	1	2	1	2	2	2
Average	2.4	2.6	2.4	2.2	1.5	-	-	-	1.4	2.2	1.2	2.2	1.8	2

References:

1. Build Your Own Security Lab, Michael Gregg, Wiley India

Ex.No. : 1a **CAESAR CIPHER**
Date :

AIM:

To implement the **Caesar Cipher algorithm** in Python to perform **encryption and decryption** of a given plaintext using a fixed shift key.

ALGORITHM :

1. Start the program.
2. Define the English alphabet (a-z).
3. Convert the input plaintext to lowercase.
4. For each character in the plaintext:
    - Find its position in the alphabet.
    - Apply the encryption formula: (position + shift) % 26.
    - Replace the character with the one at the new position.
5. Display the cipher text.
6. For decryption, repeat the process:
    - Apply the formula: (position - shift) % 26.
7. Display the decrypted text.
8. End the program.

PROGRAM (Python):

```
ALPHABET = "abcdefghijklmnopqrstuvwxyz"

def encrypt(plain_text, shift_key):
    plain_text = plain_text.lower()
    cipher_text = ""
    for char in plain_text:
        if char in ALPHABET:
            char_position = ALPHABET.index(char)
            key_val = (shift_key + char_position) % 26
            replace_val = ALPHABET[key_val]
            cipher_text += replace_val
        else:
            cipher_text += char
    return cipher_text

def decrypt(cipher_text, shift_key):
    cipher_text = cipher_text.lower()
    plain_text = ""
    for char in cipher_text:
```

```
        if char in ALPHABET:
            char_position = ALPHABET.index(char)
            key_val = (char_position - shift_key) % 26
            replace_val = ALPHABET[key_val]
            plain_text += replace_val
        else:
            plain_text += char
    return plain_text

def main():
    message = input("Enter the Plain text for Encryption: ")
    shift = 3
    encrypted = encrypt(message, shift)
    decrypted = decrypt(encrypted, shift)
    print(f"\nEncrypted message: Cipher Text = {encrypted}")
    print(f"Decrypted message: Plain Text = {decrypted}")

if __name__ == "__main__":
    main()
```

OUTPUT:

Enter the Plain text for Encryption: hello

Encrypted message: Cipher Text = khoor

Decrypted message: Plain Text = hello

RESULT:

Thus the Caesar cipher substitution technique was implemented and executed successfully.

Ex.No. : 1b **PLAYFAIR CIPHER**
Date :

AIM:

To implement the **Playfair Cipher algorithm** in Python to perform encryption and decryption of a given message using a keyword.

ALGORITHM:

1. **Input** the keyword and the message.
2. **Prepare the 5x5 matrix**:
    - Convert the keyword to uppercase and replace J with I.
    - Eliminate duplicate characters.
    - Fill the matrix with unique characters from the keyword and then the remaining letters of the alphabet (excluding J).
3. **Preprocess the message**:
    - Replace J with I.
    - Convert to uppercase and remove spaces.
    - Add 'X' between identical letters in a digraph.
    - Append 'X' at the end if the length is odd.
4. **Encryption**:
    - For each digraph:
        - If same row: take next character to the right (wrap if needed).
        - If same column: take next character below (wrap if needed).
        - Otherwise: swap column positions.
5. **Decryption**:
    - Apply reverse rules of encryption.
6. **Display** the encrypted and decrypted messages.

PYTHON PROGRAM:

```
def generate_matrix(key):
    key = "".join(dict.fromkeys(key.upper().replace("J", "I") +
"ABCDEFGHIKLMNOPQRSTUVWXYZ"))
    return [list(key[i:i+5]) for i in range(0, 25, 5)]


def find_position(matrix, char):
    for i, row in enumerate(matrix):
```

```
        if char in row:
            return i, row.index(char)
def format_message(msg):
    msg = msg.upper().replace("J", "I").replace(" ", "")
    res, i = "", 0
    while i < len(msg):
        a = msg[i]
        b = msg[i+1] if i+1 < len(msg) else "X"
        if a == b:
            res += a + "X"
            i += 1
        else:
            res += a + b
            i += 2
    return res
def encrypt_decrypt_pair(matrix, a, b, mode=1):
    r1, c1 = find_position(matrix, a)
    r2, c2 = find_position(matrix, b)
    if r1 == r2:
        return matrix[r1][(c1 + mode) % 5] + matrix[r2][(c2 + mode) % 5]
    elif c1 == c2:
        return matrix[(r1 + mode) % 5][c1] + matrix[(r2 + mode) % 5][c2]
    else:
        return matrix[r1][c2] + matrix[r2][c1]
def playfair_encrypt(msg, key):
    matrix = generate_matrix(key)
    msg = format_message(msg)
    return "".join(encrypt_decrypt_pair(matrix, msg[i], msg[i+1], 1) for i in range(0, len(msg), 2))

def playfair_decrypt(cipher, key):
```

```
    matrix = generate_matrix(key)
    return "".join(encrypt_decrypt_pair(matrix, cipher[i], cipher[i+1], -1) for i in range(0,
len(cipher), 2))
def print_matrix(matrix):
    print("Playfair Matrix:")
    for row in matrix:
        print(" ".join(row))
    print()
def main():
    key = input("Enter keyword: ")
    msg = input("Enter message to encrypt: ")
    matrix = generate_matrix(key)
    print_matrix(matrix)
    encrypted = playfair_encrypt(msg, key)
    print("Encrypted message:", encrypted)
    decrypted = playfair_decrypt(encrypted, key)
    print("Decrypted message:", decrypted)
if __name__ == "__main__":
    main()
```

SAMPLE INPUT:

Enter keyword: MONARCHY

Enter message to encrypt: INSTRUMENTS

OUTPUT:

Playfair Matrix:

M O N A R
C H Y B D
E F G I K
L P Q S T
U V W X
Encrypted message: GATLMZCLRQTX

Decrypted message: INSTRUMENTSX

RESULT:

The program successfully encrypts and decrypts a message using the **Playfair Cipher** with the given keyword.

Ex.No. : 1c **HILL CIPHER**

Date :

AIM:

To implement the **Hill Cipher algorithm** in Python using a 3×3 key matrix for encrypting and decrypting messages.

ALGORITHM:

1. **Input** the plaintext message.
2. **Convert** the message to uppercase and remove spaces.
3. **Pad** the message with 'X' to make its length a multiple of 3.
4. **Use a 3×3 key matrix** for encryption.
5. For every 3-character block:
    - Convert characters to numbers (A=0 to Z=25).
    - Multiply with the key matrix.
    - Take modulo 26 and convert numbers back to characters.
6. For decryption:
    - Use the **inverse key matrix**.
    - Follow the same steps as encryption.
7. Display both encrypted and decrypted messages.

PYTHON PROGRAM:

```
import numpy as np
# Hill Cipher Key Matrix (3x3)
key_matrix = np.array([
    [1, 2, 1],
    [2, 3, 2],
    [2, 2, 1]
])
# Pre-computed inverse of key_matrix mod 26
inv_key_matrix = np.array([
    [-1,  0,  1],
    [ 2, -1,  0],
    [-2,  2, -1]
])

alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"

def mod26(x):
    return x % 26
```

```
def text_to_vec(text):
    return [ord(c) - 65 for c in text]
def vec_to_text(vec):
    return ''.join(alphabet[mod26(i)] for i in vec)
def process_block(block, matrix):
    vec = text_to_vec(block)
    res = np.dot(matrix, vec) % 26
    return vec_to_text(res)
def prepare_text(text):
    text = text.upper().replace(" ", "")
    return text + "X" * ((3 - len(text) % 3) % 3)
def hill_encrypt(text):
    return ''.join(process_block(text[i:i+3], key_matrix) for i in range(0, len(text), 3))
def hill_decrypt(text):
    return ''.join(process_block(text[i:i+3], inv_key_matrix) for i in range(0, len(text), 3))
def main():
    plain = input("Enter the Plain text for Encryption: ")
    plain = prepare_text(plain)
    print("Padded Text:", plain)
    encrypted = hill_encrypt(plain)
    print("Encrypted Message:", encrypted)
    decrypted = hill_decrypt(encrypted)
    print("Decrypted Message:", decrypted)
if __name__ == "__main__":
    main()
```

SAMPLE INPUT:

Enter the Plain text for Encryption: ACT

SAMPLE OUTPUT:

Padded Text: ACT
Encrypted Message: GYO
Decrypted Message: ACT

RESULT:

Thus the Hill cipher substitution technique was implemented and executed successfully.

Ex.No. : 1d **VIGENERE CIPHER**
Date :

AIM:
To implement the **Vigenère Cipher** algorithm in Python to encrypt and decrypt a message using a keyword.

ALGORITHM:

1. **Input** the plaintext message and a keyword.
2. Convert both the **message and key** to uppercase.
3. For **encryption**:
    - Repeat the key to match the length of the message.
    - Shift each letter of the message by the position of the corresponding letter in the key.
    - Convert result using:
      EncryptedChar = (MessageChar + KeyChar) % 26 + 'A'
4. For **decryption**:
    - Use:
      DecryptedChar = (CipherChar - KeyChar + 26) % 26 + 'A'
5. **Display** both encrypted and decrypted text.

PYTHON PROGRAM:

```
def generate_key(message, key):
    key = key.upper()
    key = list(key)
    if len(message) == len(key):
        return "".join(key)
    else:
        for i in range(len(message) - len(key)):
            key.append(key[i % len(key)])
    return "".join(key)

def encrypt(message, key):
    message = message.upper()
    key = generate_key(message, key)
    cipher_text = ""
    for i in range(len(message)):
        if message[i].isalpha():
            x = (ord(message[i]) + ord(key[i])) % 26
```

```
            x += ord('A')
            cipher_text += chr(x)
        else:
            cipher_text += message[i]
    return cipher_text

def decrypt(cipher_text, key):
    key = generate_key(cipher_text, key)
    original_text = ""
    for i in range(len(cipher_text)):
        if cipher_text[i].isalpha():
            x = (ord(cipher_text[i]) - ord(key[i]) + 26) % 26
            x += ord('A')
            original_text += chr(x)
        else:
            original_text += cipher_text[i]
    return original_text

def main():
    key = input("Enter the key: ")
    message = input("Enter the message for encryption: ")
    encrypted = encrypt(message, key)
    decrypted = decrypt(encrypted, key)

    print("Original Message:", message)
    print("Encrypted Message:", encrypted)
    print("Decrypted Message:", decrypted)

if __name__ == "__main__":
    main()
```

SAMPLE INPUT:

Enter the key: LEMON
Enter the message for encryption: ATTACKATDAWN

OUTPUT:

Original Message: ATTACKATDAWN
Encrypted Message: LXFOPVEFRNHR
Decrypted Message: ATTACKATDAWN

RESULT:

Thus the Vigenere cipher substitution technique was implemented and executed successfully

Ex.No. : 2a **RAIL FENCE CIPHER**
Date :

Aim:

To implement the Rail Fence Cipher algorithm in Python for encrypting and decrypting text using transposition cipher technique.

Algorithm:

Encryption (Encoding):

1. Accept the plain text and depth (number of rails).
2. Create an empty matrix with depth rows and suitable number of columns.
3. Fill the matrix column-wise with the characters of the message.
4. Read the matrix row-wise to get the encrypted text.

Decryption (Decoding):

1. Accept the cipher text and depth.
2. Create an empty matrix with depth rows and suitable number of columns.
3. Fill the matrix row-wise with the cipher text.
4. Read the matrix column-wise to retrieve the original message.

Program in Python:

```
class RailFenceCipher:
    def __init__(self, depth):
        self.depth = depth
    def encrypt(self, msg):
        r = self.depth
        l = len(msg)
        c = -(-l // r)  # Ceiling division
        k = 0
        mat = [[" for _ in range(c)] for _ in range(r)]
        for i in range(c):
            for j in range(r):
                if k < l:
                    mat[j][i] = msg[k]
                    k += 1
                else:
                    mat[j][i] = 'X'
        enc = ''.join(mat[i][j] for i in range(r) for j in range(c))
        return enc
    def decrypt(self, encmsg):
        r = self.depth
```

```
        l = len(encmsg)
        c = l // r
        k = 0
        mat = [[" for _ in range(c)] for _ in range(r)]
        for i in range(r):
            for j in range(c):
                mat[i][j] = encmsg[k]
                k += 1
        dec = ''.join(mat[j][i] for i in range(c) for j in range(r))
        return dec
# Example usage
if __name__ == "__main__":
    msg = input("Enter the Plain text: ")
    depth = 2
    cipher = RailFenceCipher(depth)
    enc = cipher.encrypt(msg)
    dec = cipher.decrypt(enc)
    print("Plain Text:", msg)
    print("Encrypted Message (Cipher Text):", enc)
    print("Decrypted Message:", dec)
```

Output:

```
Enter the Plain text: WELCOME
Plain Text: WELCOME
Encrypted Message (Cipher Text): WLOECMX
Decrypted Message: WELCOMEX
```

Result:

Thus, the Rail Fence Cipher algorithm was successfully implemented in Python and the encryption and decryption processes were verified.

Ex.No. : 2b **ROW AND COLUMN TRANSFORMATION TECHNIQUE**
Date:

AIM:

To implement a Row and Column (Matrix) Transposition Technique in Python.

ALGORITHM:

1. **Input the plaintext**.
2. Remove all spaces from the text.
3. Define a fixed number of columns.
4. Calculate the number of rows as len(plaintext) // columns + (1 if remainder else 0).
5. Fill characters row-wise into a 2D matrix.
6. Read the matrix column-wise to form the cipher text.
7. To decrypt, reverse the process using the same number of rows and columns.

PROGRAM (Python):

```
def remove_spaces(text):
    return text.replace(" ", "")
def row_column_encrypt(plain_text, col):
    text = remove_spaces(plain_text)
    k = len(text)
    row = k // col + (0 if k % col == 0 else 1)
    matrix = [['#' for _ in range(col)] for _ in range(row)]
    index = 0
    for i in range(row):
        for j in range(col):
            if index < k:
                matrix[i][j] = text[index]
                index += 1
    cipher_text = ''
    for i in range(col):
        for j in range(row):
            cipher_text += matrix[j][i]
    return cipher_text, matrix
def print_matrix(matrix):
    for row in matrix:
        print(" ".join(row))

# Driver Code
plain_text = input("Enter the plain text: ")
```

```
col = 4
cipher_text, matrix = row_column_encrypt(plain_text, col)
print("\nMatrix Form:")
print_matrix(matrix)
print("\nEncrypted Cipher Text:", cipher_text)
```

SAMPLE OUTPUT:

```
Enter the plain text: altroz car shervin
Matrix Form:
a l t r
o z c a
r s h e
r v i n
Encrypted Cipher Text: aorrlzsvtchiraen
```

RESULT:

Thus, the Row and Column Transposition Technique was successfully implemented and executed in Python.

Ex.No. : 3 **DATA ENCRYPTION STANDARD (DES)**
Date :

AIM:
To implement the **Data Encryption Standard (DES)** algorithm in Python for a practical application like user message encryption and decryption.

ALGORITHM:

1. **Import required libraries**: Use pyDes or Crypto.Cipher.DES for DES implementation.
2. **Create DES key**: Use a fixed-length 8-byte key.
3. **Padding the message**: Ensure the message length is a multiple of 8 by padding if needed.
4. **Encrypt the message** using DES encryption in ECB or CBC mode.
5. **Display encrypted message** in bytes or encoded string.
6. **Decrypt the message** using the same DES key.
7. **Display the original (decrypted) message**.

PROGRAM (Python):

```
from Crypto.Cipher import DES
from Crypto.Random import get_random_bytes
import base64

# Padding for message
def pad(text):
    while len(text) % 8 != 0:
        text += ' '
    return text

# Generate a random 8-byte key
key = get_random_bytes(8)
cipher = DES.new(key, DES.MODE_ECB)

# Get user input
message = input("Enter the message to encrypt: ")
padded_message = pad(message)

# Encryption
encrypted_bytes = cipher.encrypt(padded_message.encode('utf-8'))
encrypted_base64 = base64.b64encode(encrypted_bytes).decode('utf-8')
print("Encrypted Data:", encrypted_base64)
```

```
# Decryption
decrypt_cipher = DES.new(key, DES.MODE_ECB)
decrypted_bytes = decrypt_cipher.decrypt(encrypted_bytes)
decrypted_message = decrypted_bytes.decode('utf-8').rstrip()
print("Decrypted Data:", decrypted_message)
```

OUTPUT:

```
Enter the message to encrypt:   Security Lab
Encrypted Data:  2Utsd2NlUnA1XmdVMgFvVw==
Decrypted Data: Security Lab
```

RESULT:

Thus, the DES algorithm was successfully implemented in Python for message encryption and decryption.

Ex. No.: 4 **AES ALGORITHM**
Date:

AIM:
To implement the Advanced Encryption Standard (AES) algorithm in Python for encrypting and decrypting a URL.

ALGORITHM:

1. Accept the original URL and a secret key from the user.
2. Use SHA-256 hashing to derive a consistent 256-bit key from the secret string.
3. Encrypt the URL using AES encryption with CBC mode and PKCS7 padding.
4. Encode the cipher text to Base64 for safe display or transfer.
5. Decrypt the cipher text back to the original URL using the same key and IV.
6. Display the original, encrypted, and decrypted URL.

PROGRAM (Python):

```
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad, unpad
from Crypto.Random import get_random_bytes
import base64
import hashlib

def get_key(secret_key):
    return hashlib.sha256(secret_key.encode()).digest()

def encrypt_url(url, secret_key):
    key = get_key(secret_key)
    cipher = AES.new(key, AES.MODE_CBC)
    ct_bytes = cipher.encrypt(pad(url.encode(), AES.block_size))
    iv = base64.b64encode(cipher.iv).decode('utf-8')
    ct = base64.b64encode(ct_bytes).decode('utf-8')
    return iv + ":" + ct

def decrypt_url(encrypted_data, secret_key):
    key = get_key(secret_key)
    iv, ct = encrypted_data.split(":")
    iv = base64.b64decode(iv)
    ct = base64.b64decode(ct)
    cipher = AES.new(key, AES.MODE_CBC, iv)
    pt = unpad(cipher.decrypt(ct), AES.block_size)
```

```
    return pt.decode('utf-8')

# MAIN PROGRAM
if __name__ == "__main__":
    secret_key = input("Enter the secret key: ")
    original_url = input("Enter the original URL: ")

    encrypted = encrypt_url(original_url, secret_key)
    decrypted = decrypt_url(encrypted, secret_key)

    print("\nURL Encryption Using AES Algorithm")
    print("-----------------------------------")
    print(f"Original URL   : {original_url}")
    print(f"Encrypted URL  : {encrypted}")
    print(f"Decrypted URL  : {decrypted}")
```

OUTPUT:

```
Enter the secret key: annaUniversity
Enter the original URL: www.annauniv.edu

URL Encryption Using AES Algorithm
-----------------------------------
Original URL   : www.annauniv.edu
Encrypted URL  : YVqw1rcz1x5rOtA1j+PZPA==:TjV92fZhb+8Tb6KfGUwGpA==
Decrypted URL  : www.annauniv.edu
(Note: Output will vary due to random IV generation.)
```

RESULT:

Thus, the Python program to apply the Advanced Encryption Standard (AES) algorithm for encrypting and decrypting a URL was implemented and executed successfully.

Ex. No: 5 **RSA Algorithm**

AIM:

To implement the RSA algorithm using Python

ALGORITHM:

1. Choose two prime numbers, **p** and **q**.
2. Compute the value of $\mathbf{n = p \times q}$ and $\mathbf{\phi(n) = (p - 1)(q - 1)}$.
3. Find a public key **e** such that $1 < e < \phi(n)$ and $\mathbf{gcd(e, \phi(n)) = 1}$.
4. Compute the private key **d** such that $d \equiv e^{-1} \pmod{\phi(n)}$.
5. Perform encryption and decryption:
   - **Encryption:** c = (m^e) mod n
   - **Decryption:** m = (c^d) mod n

PROGRAM:

```
def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def find_e(totient):
    for e in range(2, totient):
        if gcd(e, totient) == 1:
            return e
    raise Exception("No valid 'e' found.")

def find_d(e, totient):
    for i in range(1, 10):
        x = 1 + i * totient
        if x % e == 0:
            return x // e
    raise Exception("No valid 'd' found.")

def rsa_encrypt_decrypt(p, q, msg):
    n = p * q
    totient = (p - 1) * (q - 1)

    e = find_e(totient)        # Public exponent
```

```
    d = find_d(e, totient)      # Private key exponent

    ct = pow(msg, e, n)        # Encrypt: c = m^e mod n
    dt = pow(ct, d, n)         # Decrypt: m = c^d mod n

    return {
        'p': p,
        'q': q,
        'n': n,
        'totient': totient,
        'public_key': (e, n),
        'private_key': (d, n),
        'message': msg,
        'ciphertext': ct,
        'decrypted_message': dt
    }

# Example usage
if __name__ == "__main__":
    p = int(input("Enter first prime number (e.g. 53): "))
    q = int(input("Enter second prime number (e.g. 59): "))
    msg = int(input("Enter message as number (e.g. A=1, B=2...): "))

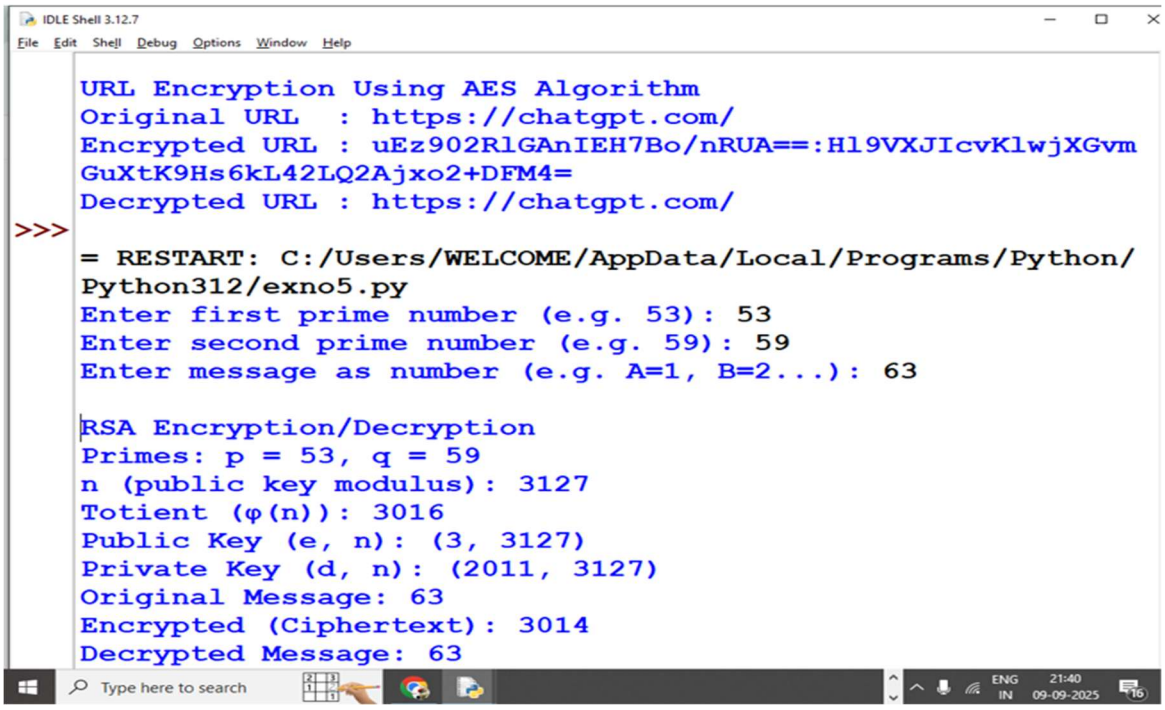
    result = rsa_encrypt_decrypt(p, q, msg)

    print("\nRSA Encryption/Decryption")
    print(f"Primes: p = {result['p']}, q = {result['q']}")
    print(f"n (public key modulus): {result['n']}")
    print(f"Totient (ϕ(n)): {result['totient']}")
    print(f"Public Key (e, n): {result['public_key']}")
    print(f"Private Key (d, n): {result['private_key']}")
    print(f"Original Message: {result['message']}")
    print(f"Encrypted (Ciphertext): {result['ciphertext']}")
    print(f"Decrypted Message: {result['decrypted_message']}")
```

Output:

RESTART: C:/Users/WELCOME/AppData/Local/Programs/Python/Python312/exno5.py
Enter first prime number (e.g. 53): 53
Enter second prime number (e.g. 59): 59
Enter message as number (e.g. A=1, B=2...): 63

RSA Encryption/Decryption
Primes: p = 53, q = 59
n (public key modulus): 3127
Totient (ϕ(n)): 3016
Public Key (e, n): (3, 3127)
Private Key (d, n): (2011, 3127)
Original Message: 63
Encrypted (Ciphertext): 3014
Decrypted Message: 63



RESULT:

Thus, the RSA algorithm was successfully implemented using Python and executed

Ex. No. : 6 **DIFFIE-HELLMAN KEY EXCHANGE ALGORITHM**
Date:

AIM:

To implement the Diffie-Hellman Key Exchange Algorithm in Python to generate a common secret key for secure communication between sender and receiver.

ALGORITHM:

1. Choose a large prime number p and a primitive root g of p.
2. Sender selects a private key x and computes R1 = g^x mod p.
3. Receiver selects a private key y and computes R2 = g^y mod p.
4. Sender and receiver exchange R1 and R2 publicly.
5. Sender computes the shared key as k1 = R2^x mod p.
6. Receiver computes the shared key as k2 = R1^y mod p.
7. Since R2^x mod p = R1^y mod p, both parties will derive the same secret key.

PROGRAM (Python):

```
def diffie_hellman(p, g, x, y):
    # Publicly shared values
    print(f"Prime number (p): {p}")
    print(f"Primitive root (g): {g}")

    # Sender side
    R1 = pow(g, x, p)
    print(f"R1 (Sender sends): {R1}")

    # Receiver side
    R2 = pow(g, y, p)
    print(f"R2 (Receiver sends): {R2}")

    # Key generation
    k1 = pow(R2, x, p)
    k2 = pow(R1, y, p)

    print(f"Key calculated at Sender's side: {k1}")
    print(f"Key calculated at Receiver's side: {k2}")

    if k1 == k2:
        print("Diffie-Hellman secret key successfully exchanged.")
    else:
```

```
        print("Key exchange failed.")

# Example usage
p = 23        # Prime number
g = 5         # Primitive root
x = 6         # Sender's private key
y = 15        # Receiver's private key

diffie_hellman(p, g, x, y)
```

OUTPUT:

```
Prime number (p): 23
Primitive root (g): 5
R1 (Sender sends): 8
R2 (Receiver sends): 2
Key calculated at Sender's side: 2
Key calculated at Receiver's side: 2
```

RESULT:

Thus, the Diffie-Hellman key exchange algorithm was successfully implemented

Ex. No.: 7 **SHA-1 ALGORITHM**
Date:

AIM:
To calculate the message digest of a text using the SHA-1 algorithm in Python.

ALGORITHM:

1. **Input the message** from the user.
2. **Encode** the message to bytes.
3. **Create** a SHA-1 digest object using the hashlib library.
4. **Update** the digest object with the byte-encoded message.
5. **Calculate** the hash using .hexdigest() method.
6. **Display** the SHA-1 hash as the message digest.

PROGRAM (Python):

```
import hashlib

def sha1_hash(message):
    sha1 = hashlib.sha1()
    sha1.update(message.encode())
    return sha1.hexdigest()

# Input from user
message = input("Enter the message: ")
digest = sha1_hash(message)

print("SHA-1 Message Digest:")
print(digest)
```

OUTPUT:

```
Enter the message: CORONA VIRUS DISEASE
SHA-1 Message Digest:
7690b7ccb987f4b3f32d2b9e7e8a69db2d0ded02
```

RESULT:
Thus, the Secure Hash Algorithm (SHA-1) was successfully implemented using Python, and the message digest was generated and verified.

Ex.No. : 8 **DIGITAL SIGNATURE SCHEME**
Date :

AIM:
To implement the signature scheme - Digital Signature Standard in Python.

ALGORITHM:

1. Choose a prime number p and calculate q, a prime factor of p-1.
2. Choose a generator g such that g = h^((p-1)/q) mod p.
3. Select private key x < q and compute public key y = g^x mod p.
4. Choose random k, compute:
    - r = (g^k mod p) mod q
    - s = (k^{-1}(h + xr)) mod q
5. Verify with:
    - w = s^{-1} mod q
    - u1 = hw mod q, u2 = rw mod q
    - v = ((g^u1)(y^u2) mod p) mod q
6. If v == r, signature is verified.
7.

PYTHON PROGRAM:

```
from sympy import isprime, nextprime, mod_inverse
import random

def get_next_prime(start_val):
    test = int(start_val)
    while not isprime(test):
        test += 1
    return test

def find_q(n):
    start = 2
    while not isprime(n):
        while n % start != 0:
            start += 1
        n = n // start
    return n

def get_gen(p, q):
    while True:
        h = random.randint(2, p-2)
```

```
        g = pow(h, (p-1)//q, p)
        if g > 1:
            return g

# Main execution
p = get_next_prime(10600)
q = find_q(p - 1)
g = get_gen(p, q)

print("\n simulation of Digital Signature Algorithm \n")
print(" \n global public key components are:\n")
print(f"p is: {p}")
print(f"q is: {q}")
print(f"g is: {g}")

x = random.randint(1, q - 1)
y = pow(g, x, p)
k = random.randint(1, q - 1)
r = pow(g, k, p) % q
h = random.randint(1, p - 1)
k_inv = mod_inverse(k, q)
s = (k_inv * (h + x * r)) % q

print("\nsecret information are:\n")
print(f"x (private) is: {x}")
print(f"k (secret) is: {k}")
print(f"y (public) is: {y}")
print(f"h (rndhash) is: {h}")

print("\n generating digital signature:\n")
print(f"r is : {r}")
print(f"s is : {s}")

# Verification
w = mod_inverse(s, q)
u1 = (h * w) % q
u2 = (r * w) % q
v = ((pow(g, u1, p) * pow(y, u2, p)) % p) % q

print("\nverifying digital signature (checkpoints):\n")
```

```
print(f"w is : {w}")
print(f"u1 is : {u1}")
print(f"u2 is : {u2}")
print(f"v is : {v}")

if v == r:
   print(f"\nsuccess: digital signature is verified!\n {r}")
else:
   print("\n error: incorrect digital signature\n ")
```

OUTPUT:

Example output from one run:
yaml
Copy code

```
simulation of Digital Signature Algorithm
global public key components are:

p is: 10601
q is: 53
g is: 6084

secret information are:

x (private) is: 7
k (secret) is: 41
y (public) is: 3204
h (rndhash) is: 8795

generating digital signature:

r is : 30
s is : 49

verifying digital signature (checkpoints):
w is : 13
u1 is : 14
u2 is : 19
v is : 30
success: digital signature is verified!
 30
```

RESULT:

Thus the Digital Signature Standard Signature Scheme has been implemented and executed successfully.

Ex. No. : 9 **INTRUSION DETECTION SYSTEM (IDS)**
Date:

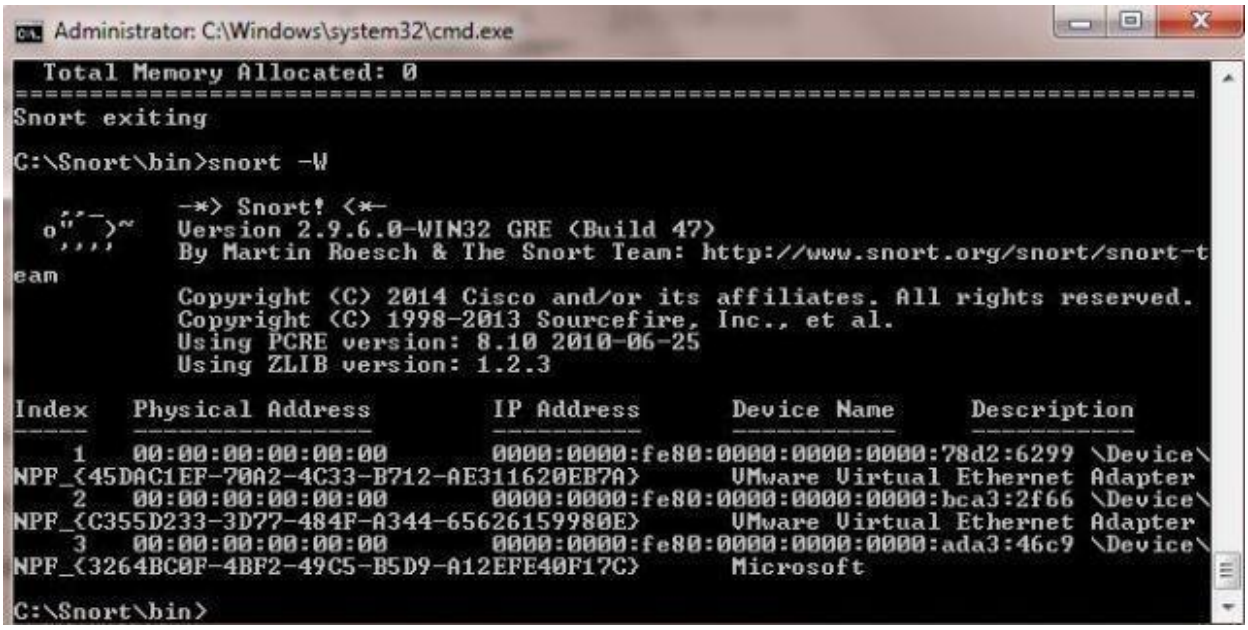
AIM:

To demonstrate Intrusion Detection System (IDS) using Snort software tool.

STEPS ON CONFIGURING AND INTRUSION DETECTION:

1. Download Snort from the Snort.org website. (http://www.snort.org/snort-downloads)
2. Download Rules(https://www.snort.org/snort-rules). You must register to get the rules. (You should download these often)
3. Double click on the .exe to install snort. This will install snort in the "C:\Snort" folder.It is important to have WinPcap (https://www.winpcap.org/install/) installed
4. Extract the Rules file. You will need WinRAR for the .gz file.
5. Copy all files from the "rules" folder of the extracted folder. Now paste the rules into *"C:\Snort\rules"* folder.
6. Copy "snort.conf" file from the "etc" folder of the extracted folder. You must paste it into "C:\Snort\etc" folder. Overwrite any existing file. Remember if you modify your snort.conf file and download a new file, you must modify it for Snort to work.
7. Open a command prompt (cmd.exe) and navigate to folder "C:\Snort\bin" folder. (at the Prompt, type cd\snort\bin)
8. To start (execute) snort in sniffer mode use following command:
snort -dev -i 3
-i indicates the interface number. You must pick the correct interface number. In my case, it is 3.
-dev is used to run snort to capture packets on your network.

To check the interface list, use following command:
snort -W



Finding an interface

You can tell which interface to use by looking at the Index number and finding Microsoft. As you can see in the above example, the other interfaces are for VMWare. My interface is 3.

9. To run snort in IDS mode, you will need to configure the file "snort.conf" according to your network environment.
10. To specify the network address that you want to protect in snort.conf file, look for the following line.
var HOME_NET 192.168.1.0/24 (You will normally see any here)
11. You may also want to set the addresses of DNS_SERVERS, if you have some on your network.

Example:

example snort
12. Change the RULE_PATH variable to the path of rules folder.
var RULE_PATH c:\snort\rules

path to rules
13. Change the path of all library files with the name and path on your system. and you must change the path of snort_dynamicpreprocessorvariable.
C:\Snort\lib\snort_dynamiccpreprocessor
You need to do this to all library files in the "C:\Snort\lib" folder. The old path might be: "/usr/local/lib/…". you will need to replace that path with your system path. Using C:\Snort\lib
14. Change the path of the "dynamicengine" variable value in the "snort.conf" file..

Example:
dynamicengine C:\Snort\lib\snort_dynamicengine\sf_engine.dll

15 Add the paths for "include classification.config" and "include reference.config" files.
include c:\snort\etc\classification.config
include c:\snort\etc\reference.config
16. Remove the comment (#) on the line to allow ICMP rules, if it is commented with a #.
include $RULE_PATH/icmp.rules
17. You can also remove the comment of ICMP-info rules comment, if it is commented.
include $RULE_PATH/icmp-info.rules
18. To add log files to store alerts generated by snort, search for the "output log" test in snort.conf and add the following line:
output alert_fast: snort-alerts.ids
19. Comment (add a #) the whitelist $WHITE_LIST_PATH/white_list.rules and the blacklist

Change the nested_ip inner , \ to nested_ip inner #, \
20. Comment out (#) following lines:
#preprocessor normalize_ip4
#preprocessor normalize_tcp: ips ecn stream
#preprocessor normalize_icmp4
#preprocessor normalize_ip6
#preprocessor normalize_icmp6

21. Save the "snort.conf" file.
22. To start snort in IDS mode, run the following command:

snort -c c:\snort\etc\snort.conf -l c:\snort\log -i 3
(Note: 3 is used for my interface card)

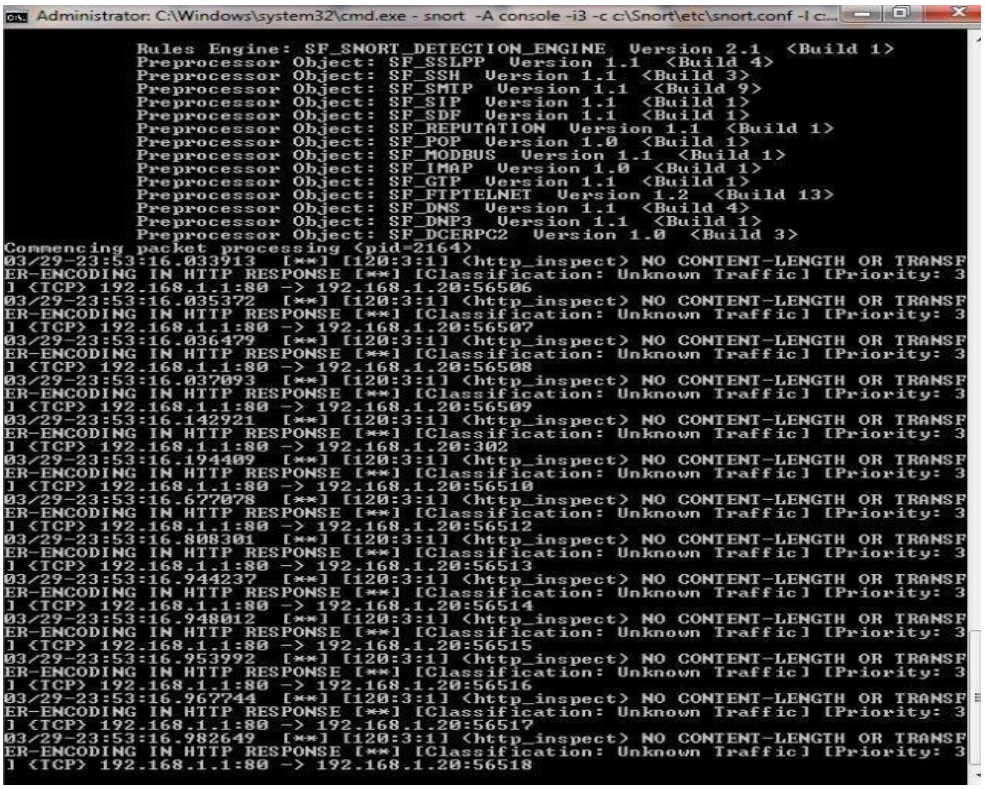
If a log is created, select the appropriate program to open it. You can use WordPard or NotePad++ to read the file.

To generate Log files in ASCII mode, you can use following command while running snort in IDS mode:
snort -A console -i3 -c c:\Snort\etc\snort.conf -l c:\Snort\log -K ascii

23. Scan the computer that is running snort from another computer by using PING or NMap (ZenMap).

After scanning or during the scan you can check the snort-alerts.ids file in the log folder to insure it is logging properly. You will see IP address folders appear.

Snort monitoring traffic –



RESULT:

Thus the Intrusion Detection System(IDS) has been demonstrated using the Open Source Intrusion Detection Tool Snort.

Ex.No. : 10 **EXPLORING N-STALKER**
Date :

AIM:

To download the N-Stalker Vulnerability Assessment Tool and exploring the features.

EXPLORING N-STALKER:

- N-Stalker Web Application Security Scanner is a Web security assessment tool.
- It incorporates with a well-known N-Stealth HTTP Security Scanner and 35,000 Web attack signature database.
- This tool also comes in both free and paid version.
- Before scanning the target, go to "License Manager" tab, perform the update.
- Once update, you will note the status as up to date.
- You need to download and install N-Stalker from www.nstalker.com.

1. Start N-Stalker from a Windows computer. The program is installed under Start ➪ Programs ➪ N-Stalker ➪ N-Stalker Free Edition.
2. Enter a host address or a range of addresses to scan.
3. Click Start Scan.
4. After the scan completes, the N-Stalker Report Manager will prompt
5. you to select a format for the resulting report as choose Generate HTML.
6. Review the HTML report for vulnerabilities.





Now goto "Scan Session", enter the target URL.

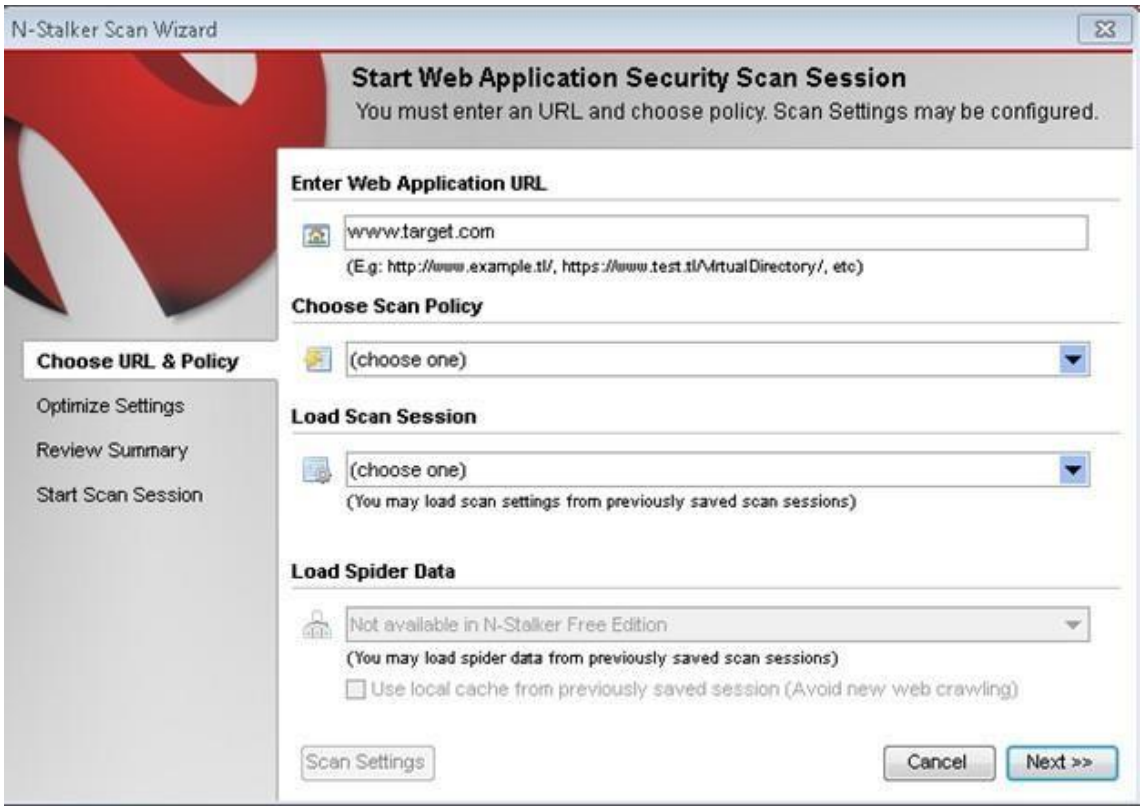
In scan policy, you can select from the four options,

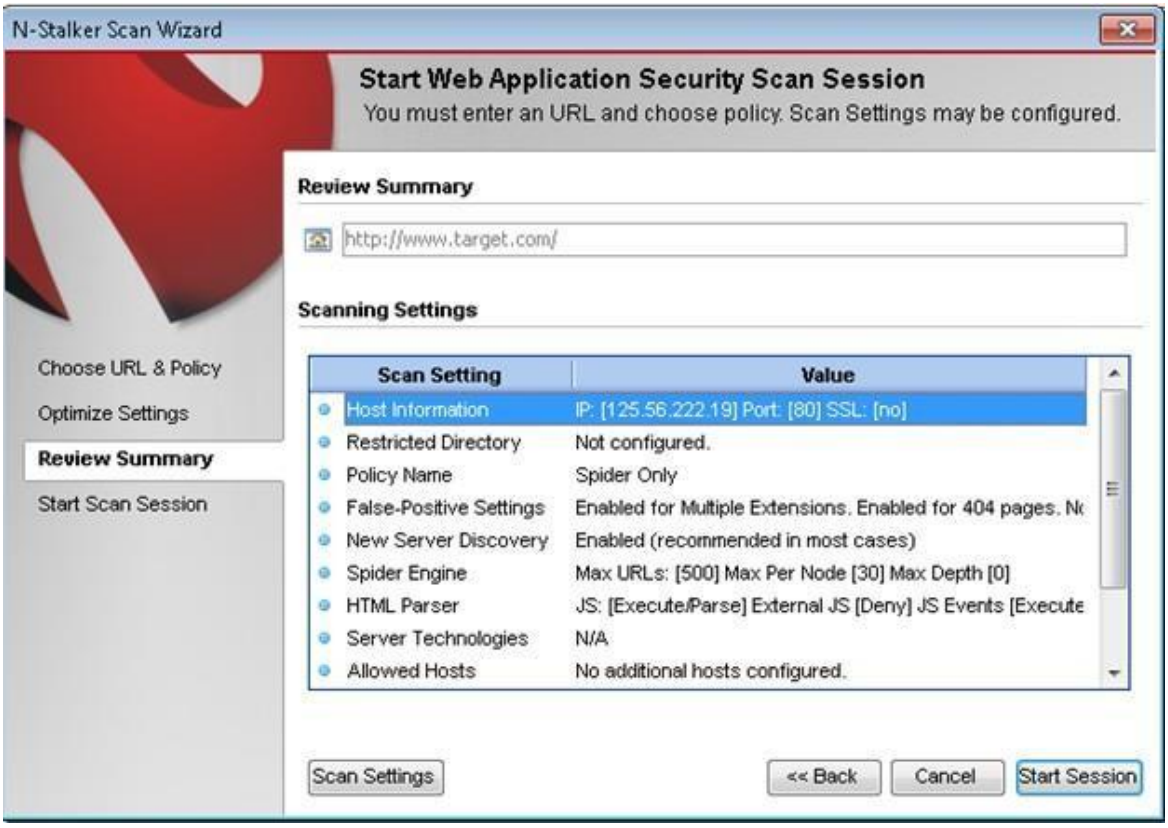
- Manual test which will crawl the website and will be waiting for manual attacks.
- full xss assessment
- owasp policy

- Web server infrastructure analysis.

Once, the option has been selected, next step is "Optimize settings" which will crawl the whole website for further analysis.

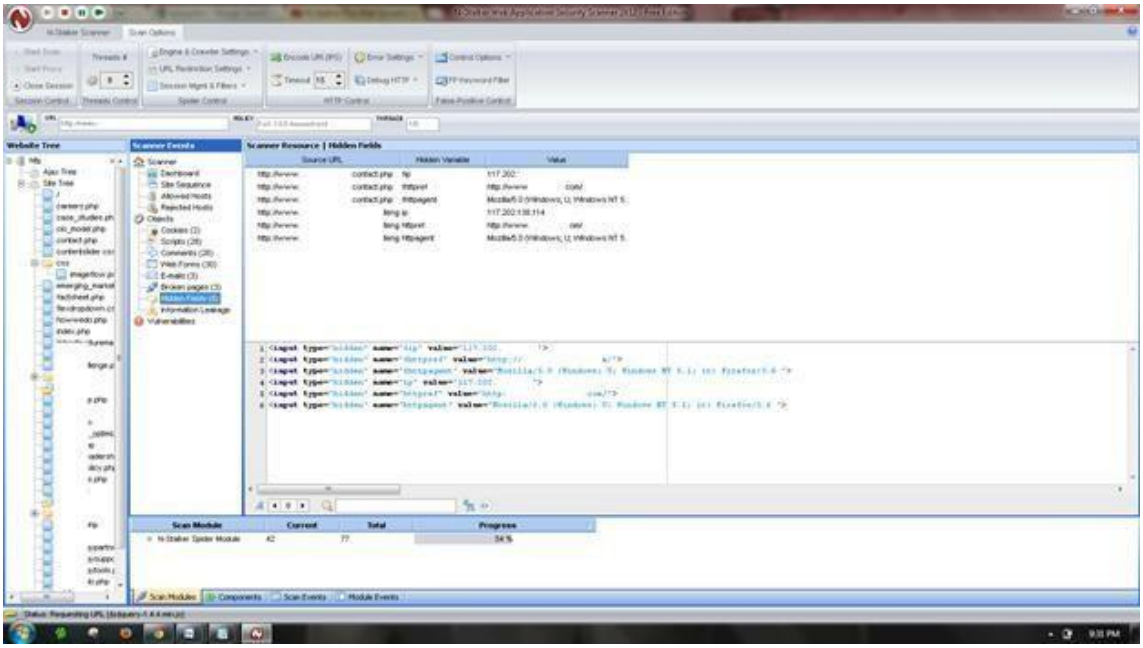
In review option, you can get all the information like host information, technologies used, policy name, etc.



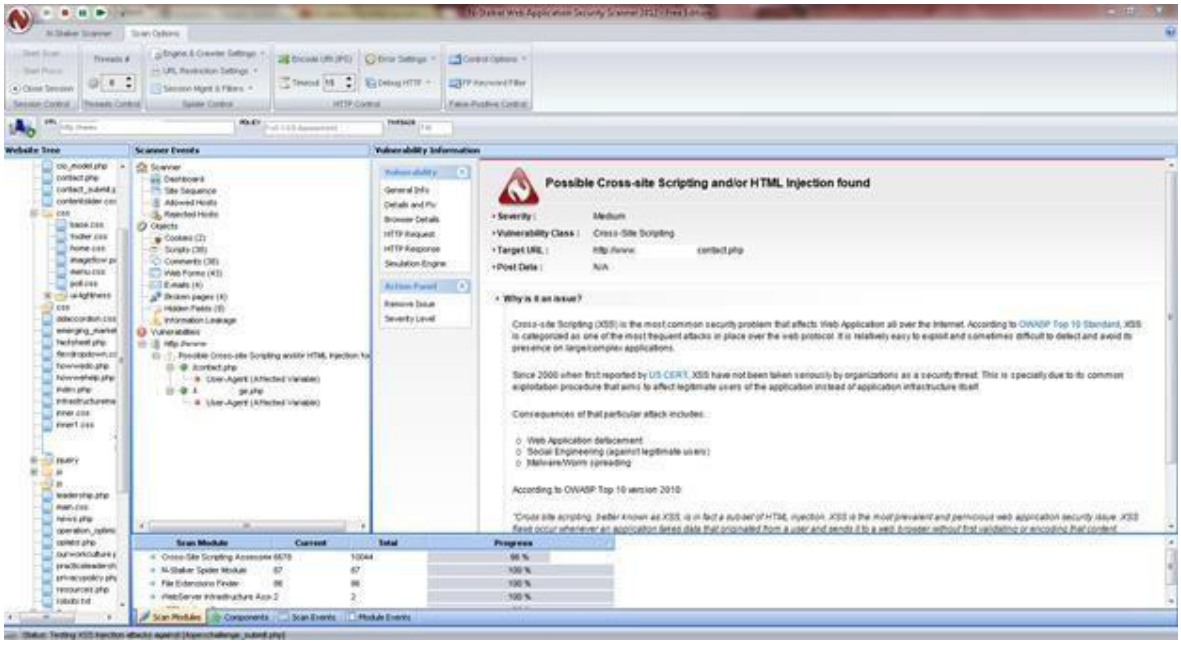


Once done, start the session and start the scan.

The scanner will crawl the whole website and will show the scripts, broken pages, hidden fields, information leakage, web forms related information which helps to analyze further.



Once the scan is completed, the NStalker scanner will show details like severity level, vulnerability class, why is it an issue, the fix for the issue and the URL which is vulnerable to the particular vulnerability?



RESULT:

Thus the N-Stalker Vulnerability Assessment tool has been downloaded, installed and the features has been explored by using a vulnerable website.

Ex.No.: 11a **DEFEATING MALWARE - BUILDING TROJANS**
Date :

AIM:

To build a Trojan and know the harmness of the Trojan malwares in a computer system.

PROCEDURE:

1. Create a simple Trojan by using Windows Batch File (***.bat***)
2. Type these below code in notepad and save it as **Trojan.bat**
3. Double click on ***Trojan.bat*** file.
4. When the Trojan code executes, it will open MS-Paint, Notepad, Command Prompt, Explorer, etc., infinitely.
5. Restart the computer to stop the execution of this Trojan.

TROJAN:

- In computing, a Trojan horse,or Trojan, is any malware which misleads users of its true intent.
- Trojans are generally spread by some form of social engineering, for example where a user is duped into executing an email attachment disguised to appear not suspicious, (e.g., a routine form to be filled in), or by clicking on some fake advertisement on social media or anywhere else.
- Although their payload can be anything, many modern forms act as a backdoor, contacting a controller which can then have unauthorized access to the affected computer.
- Trojans may allow an attacker to access users' personal information such as banking information, passwords, or personal identity.
- ***Example:*** *Ransomware* attacks are often carried out using a *trojan*.

CODE:

Trojan.bat

```
@echo off
:x
start mspaint
start notepad
start cmd
start explorer
start control
start calc
goto x
```

OUTPUT:

(MS-Paint, Notepad, Command Prompt, Explorer will open infinitely)

RESULT:

Thus a trojan has been built and the harmness of the trojan viruses has been explored.

Ex.No. : 11b **DEFEATING MALWARE - ROOTKIT HUNTER**
Date :

AIM:

To install a rootkit hunter and find the malwares in a computer.

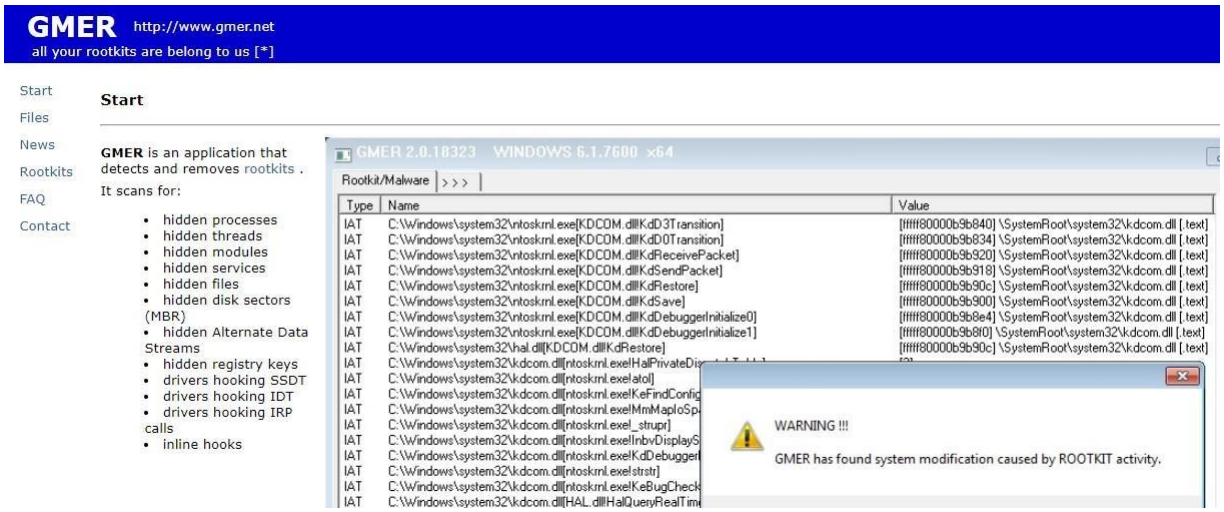
ROOTKIT HUNTER:

- rkhunter (Rootkit Hunter) is a Unix-based tool that scans for rootkits, backdoors and possible local exploits.
- It does this by comparing SHA-1 hashes of important files with known good ones in online databases, searching for default directories (of rootkits), wrong permissions, hidden files, suspicious strings in kernel modules, and special tests for Linux and FreeBSD.
- rkhunter is notable due to its inclusion in popular operating systems (Fedora, Debian, etc.)
- The tool has been written in Bourne shell, to allow for portability. It can run on almost all UNIX-derived systems.

GMER ROOTKIT TOOL:

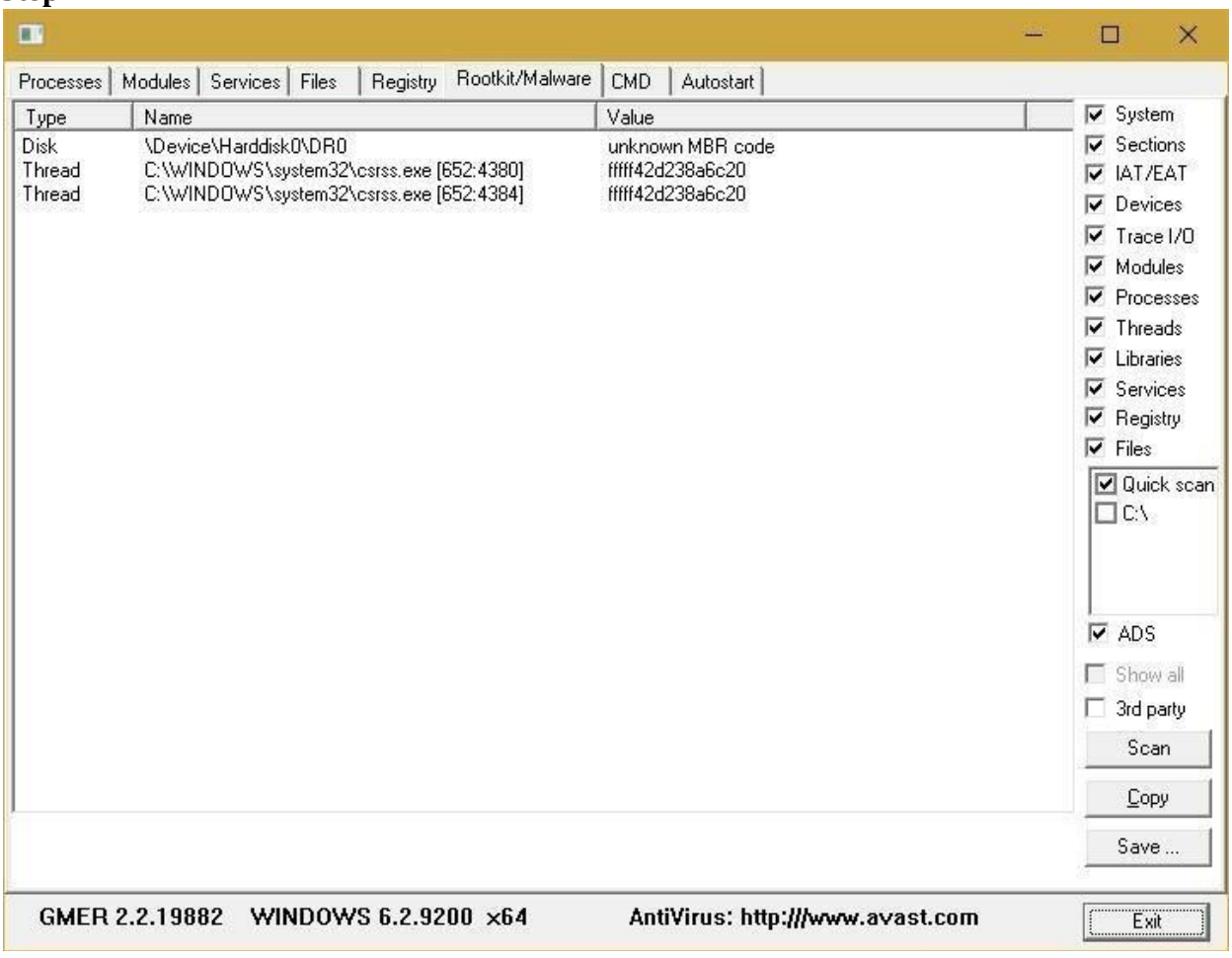
- GMER is a software tool written by a Polish researcher Przemysław Gmerek, for detecting and removing rootkits.
- It runs on Microsoft Windows and has support for Windows NT, 2000, XP, Vista, 7, 8 and 10. With version 2.0.18327 full support for Windows x64 is added.

Step 1



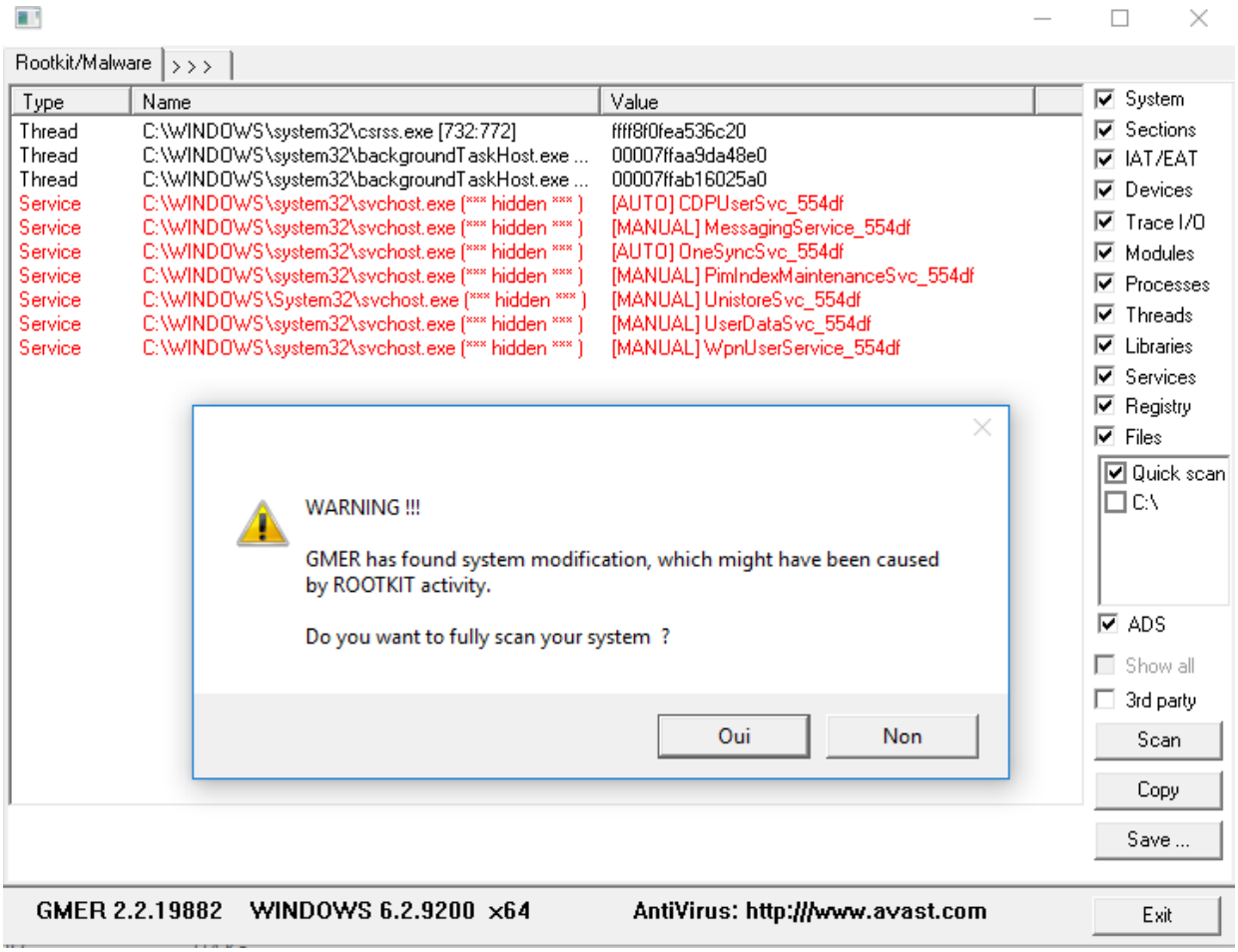
Visit GMER's website (see Resources) and download the GMER executable.
Click the "Download EXE" button to download the program with a random file name, as some rootkits will close “gmer.exe” before you can open it.

Step 2



Double-click the icon for the program.
Click the "Scan" button in the lower-right corner of the dialog box. Allow the program to scan your entire hard drive.

Step 3



When the program completes its scan, select any program or file listed in red. Right-click it and select "Delete."
If the red item is a service, it may be protected. Right-click the service and select "Disable." Reboot your computer and run the scan again, this time selecting "Delete" when that service is detected.
When your computer is free of Rootkits, close the program and restart your PC.

RESULT:

A rootkit hunter software tool *gmer* has been installed and the rootkits have been detected.